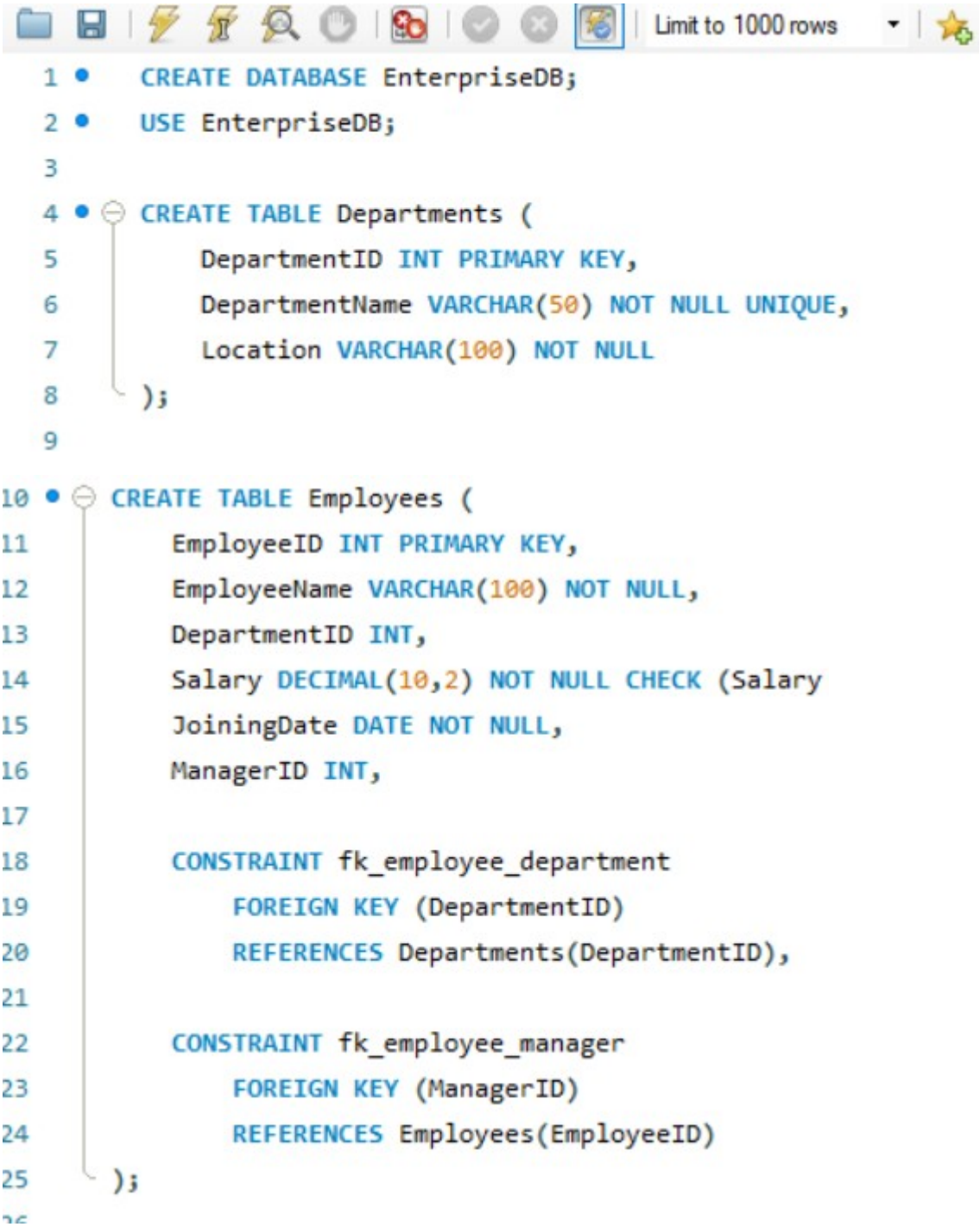
Hcltech assessment

Name: Anmol Janamatti

USN: 2JI23CS022

Date: 28/10/2026

Question 1: Database Design and Advanced Queries



```sql
INSERT INTO Departments
    (DepartmentID, DepartmentName, Location)
VALUES
    (1, 'IT', 'Bengaluru'),
    (2, 'HR', 'Mumbai'),
    (3, 'Finance', 'Delhi'),
    (4, 'Marketing', 'Chennai'),
    (5, 'Research', 'Hyderabad');

    INSERT INTO Employees
    (EmployeeID, EmployeeName, DepartmentID,
     Salary, JoiningDate, ManagerID)
VALUES
    (102, 'Neha',    1, 70000, '2021-04-10', 101),
    (103, 'Arjun',   1, 70000, '2022-02-18', 101),
    (105, 'Karan',   2, 50000, '2022-08-01', 104),
    (106, 'Pooja',   2, 45000, '2023-01-15', 104),
    (108, 'Vikram',  3, 60000, '2020-11-25', 107);
```

```sql
INSERT INTO Employees
(EmployeeID, EmployeeName, DepartmentID,
 Salary, JoiningDate, ManagerID)
VALUES
    (101, 'Amit',    1, 90000, '2020-01-15', NULL),
    (104, 'Priya',   2, 75000, '2019-06-10', NULL),
    (107, 'Rahul',   3, 85000, '2018-03-20', NULL),
    (110, 'Sneha',   4, 70000, '2021-07-12', NULL);
```

a. Display employee names, department names, and manager names using joins.

Answer:

```
57 •  SELECT
58        e.EmployeeName AS EmployeeName,
59        d.DepartmentName AS DepartmentName,
60        m.EmployeeName AS ManagerName
61    FROM Employees e
62    LEFT JOIN Departments d
63        ON e.DepartmentID = d.DepartmentID
64    LEFT JOIN Employees m
65        ON e.ManagerID = m.EmployeeID
66    ORDER BY e.EmployeeID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeName	DepartmentName	ManagerName
Amit	IT	NULL
Neha	IT	Amit
Arjun	IT	Amit
Priya	HR	NULL
Karan	HR	Priya
Pooja	HR	Priya
Rahul	Finance	NULL
Vikram	Finance	Rahul

Result 6

b. Find employees earning more than their department's average salary.

Answer:

```
77
78      FROM Employees e
79      WHERE e.Salary > (
80          SELECT AVG(e2.Salary)
81          FROM Employees e2
82          WHERE e2.DepartmentID = e.DepartmentID
83      )
84      ORDER BY e.DepartmentID, e.Salary DESC;
85
86
```

Result Grid | Filter Rows: | Edit: | Export/Import:

EmployeeID	EmployeeName	DepartmentID	Salary
101	Amit	1	90000.00
104	Priya	2	75000.00
107	Rahul	3	85000.00
NULL	NULL	NULL	NULL

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

Answer:

```
91 •    SELECT MAX(Salary) AS SecondHighestSalary
92      FROM Employees
93      WHERE Salary < (
94          SELECT MAX(Salary)
95          FROM Employees
96      );
97
98
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

SecondHighestSalary
85000.00

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

Answer:

```
103  WITH RankedEmployees AS (
104      SELECT
105          EmployeeID,
106          EmployeeName,
107          DepartmentID,
108          Salary,
109          RANK() OVER (
110              PARTITION BY DepartmentID
111              ORDER BY Salary DESC
112          ) AS SalaryRank
113      FROM Employees
114  )
```

```
115  SELECT
116      EmployeeID,
117      EmployeeName,
118      DepartmentID,
119      Salary,
120      SalaryRank
121  FROM RankedEmployees
122  WHERE SalaryRank <= 3
123  ORDER BY DepartmentID, Salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	EmployeeName	DepartmentID	Salary	SalaryRank
101	Amit	1	90000.00	1
102	Neha	1	70000.00	2
103	Arjun	1	70000.00	2
104	Priya	2	75000.00	1
105	Karan	2	50000.00	2
106	Pooja	2	45000.00	3
107	Rahul	3	85000.00	1
108	Vikram	3	60000.00	2

Question 2: Sales Analytics and Query Optimization




```
create database saletest;
use saletest;

CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    SaleDate DATE NOT NULL,
    ProductName VARCHAR(100) NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice >= 0)
);


INSERT INTO Sales
    (SaleID, CustomerID, SaleDate, ProductName, Quantity, UnitPrice)
VALUES
    (1,  101, '2025-01-05', 'Laptop',   2, 55000.00),
    (2,  102, '2025-01-10', 'Mouse',    5,   500.00),
    (3,  103, '2025-01-15', 'Keyboard', 3,  1500.00),
    (4,  101, '2025-02-03', 'Laptop',   1, 55000.00),
    (5,  104, '2025-02-12', 'Mouse',    4,   500.00),
    (6,  102, '2025-02-20', 'Monitor',  2, 12000.00),
    (7,  105, '2025-03-02', 'Keyboard', 2,  1500.00);
```

a. Calculate total revenue for each product.

Answer:

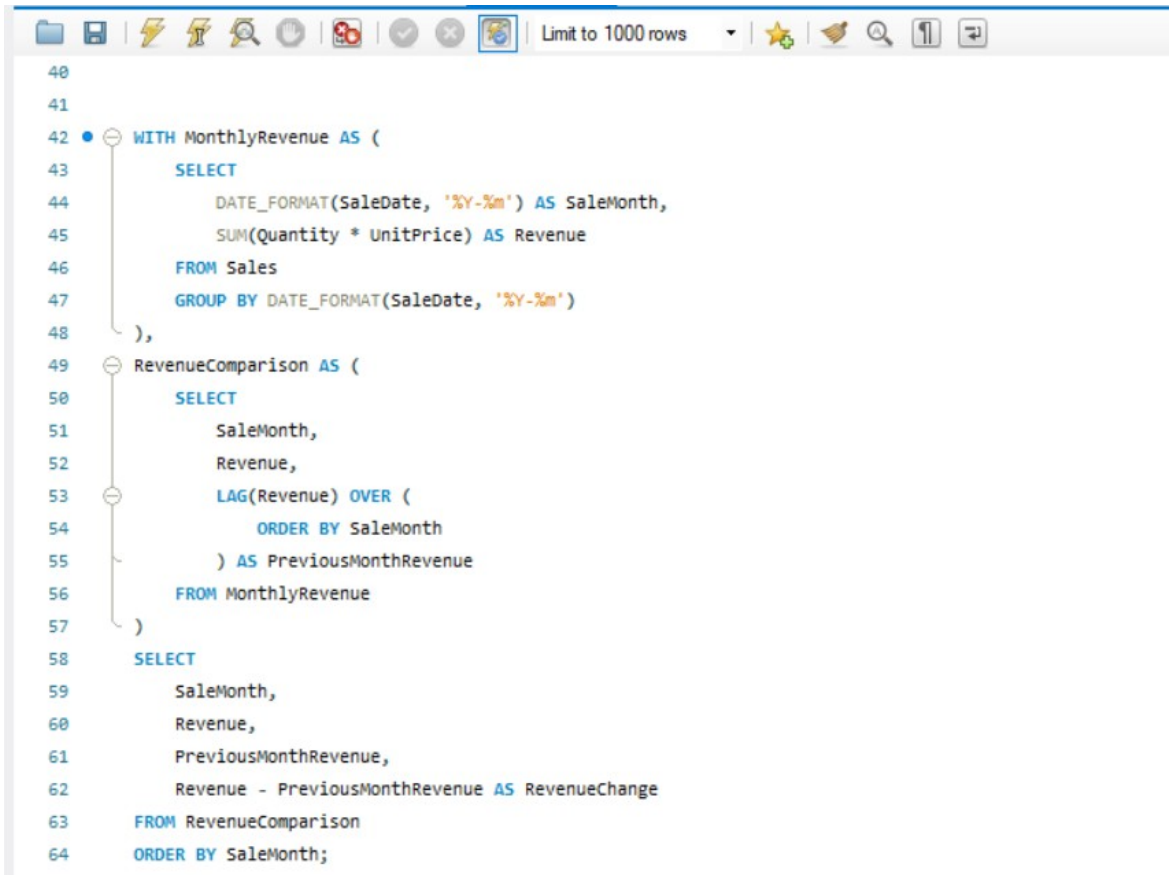
```
28
29 •        SELECT
30          ProductName,
31          SUM(Quantity * UnitPrice) AS TotalRevenue
32      FROM Sales
33      GROUP BY ProductName
34      ORDER BY TotalRevenue DESC;
35
36
37
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
Laptop	165000.00
Monitor	24000.00
Keyboard	7500.00
Mouse	4500.00

b. Calculate monthly revenue and month-over-month revenue change using a window function.

Answer:



```
40
41
42  WITH MonthlyRevenue AS (
43      SELECT
44          DATE_FORMAT(SaleDate, '%Y-%m') AS SaleMonth,
45          SUM(Quantity * UnitPrice) AS Revenue
46      FROM Sales
47      GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
48  ),
49  RevenueComparison AS (
50      SELECT
51          SaleMonth,
52          Revenue,
53          LAG(Revenue) OVER (
54              ORDER BY SaleMonth
55          ) AS PreviousMonthRevenue
56      FROM MonthlyRevenue
57  )
58  SELECT
59      SaleMonth,
60      Revenue,
61      PreviousMonthRevenue,
62      Revenue - PreviousMonthRevenue AS RevenueChange
63  FROM RevenueComparison
64  ORDER BY SaleMonth;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SaleMonth	Revenue	PreviousMonthRevenue	RevenueChange
2025-01	117000.00	NULL	NULL
2025-02	81000.00	117000.00	-36000.00
2025-03	3000.00	81000.00	-78000.00

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better.

SELECT * FROM Employees WHERE YEAR(JoiningDate) = 2025; Assume JoiningDate may be indexed.

Answer:

```
SELECT *
FROM Employees
WHERE JoiningDate >= '2025-01-01'
  AND JoiningDate < '2026-01-01';
```

Question 3: SQL Troubleshooting

(a) Explain how an execution plan helps identify expensive operations in a query.

Answer:

An execution plan shows how a database executes a query. It helps identify full table scans, inefficient index usage, costly joins, and sorting operations. It also provides estimated costs and row counts to help optimize query performance.

Example: EXPLAIN SELECT * FROM Employees WHERE DepartmentID = 10;

(b) Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

Answer:

A composite index on DepartmentID and Salary is recommended because it improves filtering by department and salary.

SQL Query: CREATE INDEX idx_emp_department_salary ON Employees (DepartmentID, Salary);

This index helps the database locate employees in a specific department and filter them by salary, reducing the number of rows examined.

(c) Write a query to identify duplicate customer email addresses and display only addresses occurring more than once.

Answer:

SQL Query:

SELECT Email, COUNT*() AS Occurrences*
FROM Customers
WHERE Email IS NOT NULL
GROUP BY Email
HAVING COUNT() > 1;

This query groups identical email addresses, counts their occurrences, and displays only those appearing more than once.

Linux

Question 4: Log Analysis

(a) Find all lines containing ERROR.

Answer:

grep 'ERROR' /var/log/application.log

(b) Display the 20 most recent error entries in the file.

Answer:

grep 'ERROR' /var/log/application.log | tail -n 20

(c) Count the total number of error entries.

Answer:

grep -c 'ERROR' /var/log/application.log

(d) Extract error entries for a specified date into a separate file.

Answer:

grep '^2026-10-28 .*ERROR' /var/log/application.log > errors_2026-10-28.log

(e) Explain how you would calculate the number of errors occurring in each hour.

Answer:

Use awk to extract the date and hour from each error entry and count the occurrences for each hour.

awk '$3 == "ERROR" { hour = substr($2,1,2); count[hour]++ } END { for (h in count) print h ":00 - " count[h] " errors" }' /var/log/application.log

This command assumes the log format has separate date and time fields followed by the log level. It counts error entries for each hour.

Question 5: Permissions and System Troubleshooting

A Linux server hosts an application that processes confidential financial reports.

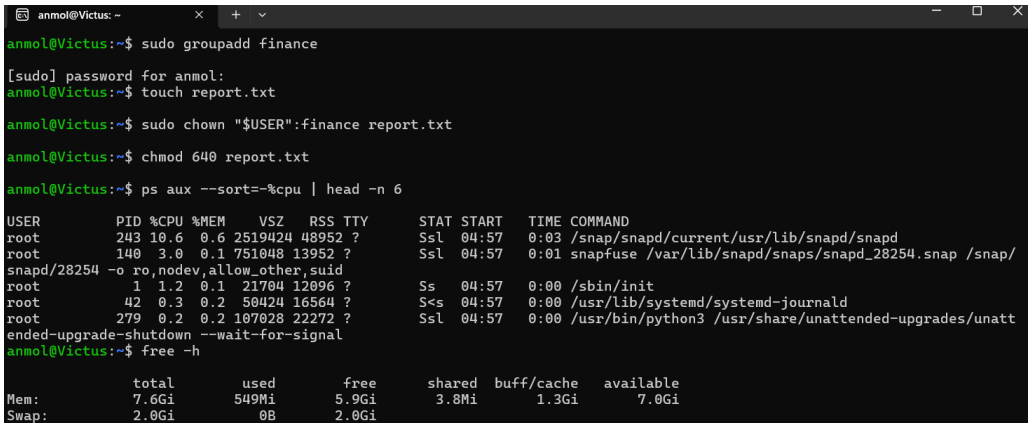
Perform the following tasks:

(a) Create a group named finance and a report file, assuming you have the required privileges.

(b) Set permissions so the owner can read and write, the finance group can read, and other users have no access.

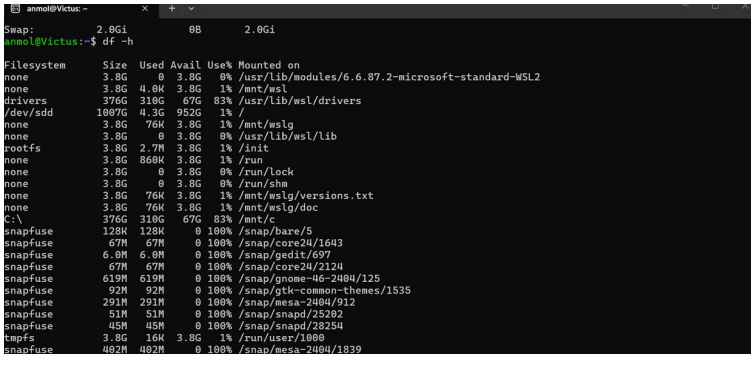
(c) Identify the processes consuming the most CPU.

Answer:



(d) Display memory usage and filesystem disk utilization.

Answer:



(e) Find files larger than 100 MB under a specified application directory.

Answer:

```
anmol@Victus:~$ find . -type f -size +100M


./.nvm/versions/node/v24.21.0/bin/node
./.hermes/node/bin/node
anmol@Victus:~$
```

Question 6: Bash Scripting

Write a Bash script named backup_check.sh that:

1. Accepts source and destination directories as command-line arguments.

2. Validates that both arguments are provided.

3. Checks whether the source directory exists.

4. Creates the destination directory if it does not exist.

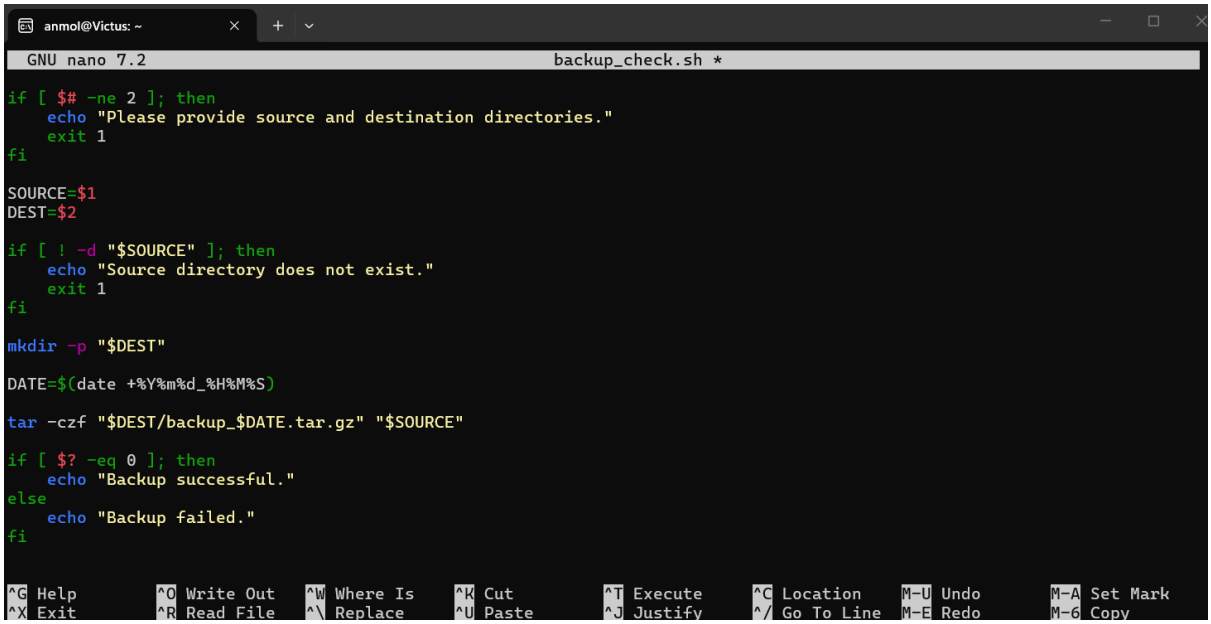
5. Creates a timestamped compressed archive of the source directory.

6. Displays a success message if the backup succeeds.

7. Displays an appropriate error message if the backup fails.

Expected Output: Submit the complete script and demonstrate its execution using valid and invalid arguments.

Answer:



```
anmol@Victus:~$ nano backup_check.sh
anmol@Victus:~$ chmod +x backup_check.sh
anmol@Victus:~$
anmol@Victus:~$ mkdir -p ~/source_data
anmol@Victus:~$ echo "Hello Backup" > ~/source_data/test.txt
anmol@Victus:~$ ./backup_check.sh ~/source_data ~/backups
Backup successful: /home/anmol/backups/backup_20261028_051348.tar.gz
anmol@Victus:~$ ./backup_check.sh
Error: Please provide source and destination directories.
Usage: ./backup_check.sh <source_directory> <destination_directory>
anmol@Victus:~$ ./backup_check.sh ~/missing ~/backups
Error: Source directory does not exist.
anmol@Victus:~$ ls ~/backups

backup_20261028_051348.tar.gz
anmol@Victus:~$
```

Question 7: PySpark DataFrame Operations

Create a PySpark DataFrame containing the following columns: EmployeeID, EmployeeName, Department, Salary, JoiningDate. Insert at least 15 sample records.

Write and execute a PySpark program to:

a. Display the schema and first five records.

b. Filter employees whose monthly salary is greater than ₹60,000.

c. Add an AnnualSalary column calculated from the monthly salary.

d. Group employees by department and calculate the average salary and employee count.

e. Display the five highest-paid employees.

Expected Output: Submit the complete PySpark program and execution results.

Answer:

[1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName("Question7") \
    .master("local[*]") \
    .getOrCreate()

data = [
    (1, "Amit", "IT", 75000, "2022-01-10"),
    (2, "Priya", "HR", 45000, "2021-03-15"),
    (3, "Rahul", "IT", 85000, "2020-06-20"),
    (4, "Sneha", "Finance", 65000, "2023-02-01"),
    (5, "Kiran", "HR", 55000, "2022-08-12"),
    (6, "Neha", "IT", 92000, "2019-11-05"),
    (7, "Arjun", "Finance", 70000, "2021-07-18"),
    (8, "Divya", "IT", 60000, "2023-04-10")
]

columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]

df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))

print("a. Schema:")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)

print("b. Employees earning more than 60000:")
df.filter(col("Salary") > 60000).show()

print("c. Annual Salary:")
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.select("EmployeeName", "Salary", "AnnualSalary").show()

print("d. Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()

print("e. Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5)
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0.
Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
a. Schema:
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

First five records:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|1         |Amit        |IT        |75000 |2022-01-10 |
|2         |Priya       |HR        |45000 |2021-03-15 |
|3         |Rahul       |IT        |85000 |2020-06-20 |
|4         |Sneha       |Finance   |65000 |2023-02-01 |
|5         |Kiran       |HR        |55000 |2022-08-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
b. Employees earning more than 60000:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 75000| 2022-01-10|
|         3|       Rahul|        IT| 85000| 2020-06-20|
|         4|       Sneha|   Finance| 65000| 2023-02-01|
|         6|        Neha|        IT| 92000| 2019-11-05|
|         7|       Arjun|   Finance| 70000| 2021-07-18|
+----------+------------+----------+------+-----------+

c. Annual Salary:
+------------+------+------------+
|EmployeeName|Salary|AnnualSalary|
+------------+------+------------+
|        Amit| 75000|      900000|
|       Priya| 45000|      540000|
|       Rahul| 85000|     1020000|
|       Sneha| 65000|      780000|
|       Kiran| 55000|      660000|
|        Neha| 92000|     1104000|
|       Arjun| 70000|      840000|
|       Divya| 60000|      720000|
+------------+------+------------+

d. Department-wise average salary and employee count:
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|        IT|      78000.0|            4|
|        HR|      50000.0|            2|
|   Finance|      67500.0|            2|
+----------+-------------+-------------+

e. Five highest-paid employees:
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         6|        Neha|        IT| 92000| 2019-11-05|     1104000|
|         3|       Rahul|        IT| 85000| 2020-06-20|     1020000|
|         1|        Amit|        IT| 75000| 2022-01-10|      900000|
|         7|       Arjun|   Finance| 70000| 2021-07-18|      840000|
|         4|       Sneha|   Finance| 65000| 2023-02-01|      780000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation

A company receives a CSV file named transactions.csv containing: TransactionID, CustomerID, TransactionDate, Product, Quantity, UnitPrice. The file contains duplicate records, missing values, and incorrectly formatted dates.

Write a PySpark program to:

a. Read the CSV file using an appropriate schema and header configuration.

b. Identify and remove duplicate transactions using a suitable business key.

c. Handle missing values in required fields and explain your chosen approach.

d. Convert TransactionDate into a date type and handle invalid dates.

e. Calculate revenue using Quantity × UnitPrice.

f. Calculate total revenue by product and display the results in descending order.

Expected Output: Submit the program and show the cleaned data and aggregation results.

Answer:

```
[4]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date, sum
from pyspark.sql.functions import expr


spark = SparkSession.builder.appName("DataCleaning").getOrCreate()

df = spark.read.csv("transactions.csv", header=True, inferSchema=True)

print("Original Data:")
df.show()

# Remove duplicates
df = df.dropDuplicates(["TransactionID"])


df = df.withColumn(
    "TransactionDate",
    expr("try_to_date(TransactionDate, 'yyyy-MM-dd')")
)
```

```
df = df.filter(
    col("CustomerID").isNotNull() &
    col("TransactionDate").isNotNull() &
    col("Quantity").isNotNull() &
    col("UnitPrice").isNotNull()
)

df = df.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

print("Cleaned Data:")
df.show()

print("Revenue by Product:")
df.groupBy("Product").agg(
    sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc()).show()
```

```
Original Data:
+-------------------+----------+---------------+--------+--------+---------+
|      TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------------+----------+---------------+--------+--------+---------+
|2026-10-28 01:00:00|        C1|     2026-01-10|  Laptop|       1|    50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 03:00:00|        C3|        invalid|Keyboard|       1|     1500|
|2026-10-28 04:00:00|      NULL|     2026-01-12|  Laptop|       1|    50000|
+-------------------+----------+---------------+--------+--------+---------+

Cleaned Data:
+-------------------+----------+---------------+-------+--------+---------+-------+
|      TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------------+----------+---------------+-------+--------+---------+-------+
|2026-10-28 01:00:00|        C1|     2026-01-10| Laptop|       1|    50000|  50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|  Mouse|       2|      500|   1000|
+-------------------+----------+---------------+-------+--------+---------+-------+

Revenue by Product:
+-------+------------+
|Product|TotalRevenue|
+-------+------------+
| Laptop|       50000|
|  Mouse|        1000|
+-------+------------+
```

Question 9: PySpark Joins and Performance

A company maintains two datasets: Customers (CustomerID, CustomerName, City) and Orders (OrderID, CustomerID, OrderAmount, OrderDate).

Write a PySpark program to:

a. Load both datasets into DataFrames.

Answer:

```
[1]: from pyspark.sql import SparkSession
     from pyspark.sql.functions import col, sum, broadcast

     spark = SparkSession.builder \
         .appName("CustomerOrders") \
         .getOrCreate()

     customers_data = [
         (1, "Amit", "Bengaluru"),
         (2, "Priya", "Mumbai"),
         (3, "Rahul", "Delhi"),
         (4, "Sneha", "Bengaluru"),
         (5, "Kiran", "Chennai")
     ]

     orders_data = [
         (101, 1, 5000.0, "2026-01-10"),
         (102, 2, 3000.0, "2026-01-12"),
         (103, 1, 2000.0, "2026-01-15"),
         (104, 3, 7000.0, "2026-01-18"),
         (105, 4, 4000.0, "2026-01-20")
     ]

     customers = spark.createDataFrame(
         customers_data,
         ["CustomerID", "CustomerName", "City"]
     )

     orders = spark.createDataFrame(
         orders_data,
         ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
     )

     print("Customers:")
     customers.show()

     print("Orders:")
     orders.show()
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing
0. Some features may not work correctly. It is recommended to use pa
  require_minimum_pandas_version()
Customers:
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|        Amit|Bengaluru|
|         2|       Priya|   Mumbai|
|         3|       Rahul|    Delhi|
|         4|       Sneha|Bengaluru|
|         5|       Kiran|  Chennai|
+----------+------------+---------+

Orders:
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|     5000.0|2026-01-10|
|    102|         2|     3000.0|2026-01-12|
|    103|         1|     2000.0|2026-01-15|
|    104|         3|     7000.0|2026-01-18|
|    105|         4|     4000.0|2026-01-20|
+-------+----------+-----------+----------+
```

b. Join the datasets to display orders with customer details.

Answer:

```
joined = orders.join(customers, "CustomerID", "inner")

print("Orders with customer details:")
joined.show()
```

```
Orders with customer details:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

c. Identify customers who have never placed an order.

Answer:

```
[3]: print("Customers without orders:")

customers.join(
    orders.select("CustomerID").distinct(),
    "CustomerID",
    "left_anti"
).show()
```

```
Customers without orders:
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|         5|       Kiran|Chennai|
+----------+------------+-------+
```

d. Calculate the total order value by city.

e. Explain when a broadcast join is useful and what conditions make it appropriate.

Answer:

```
print("Broadcast join:")

broadcast(orders).join(customers, "CustomerID").show()
```

```
Broadcast join:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

f. Explain the difference between transformations and actions, giving one example of each.

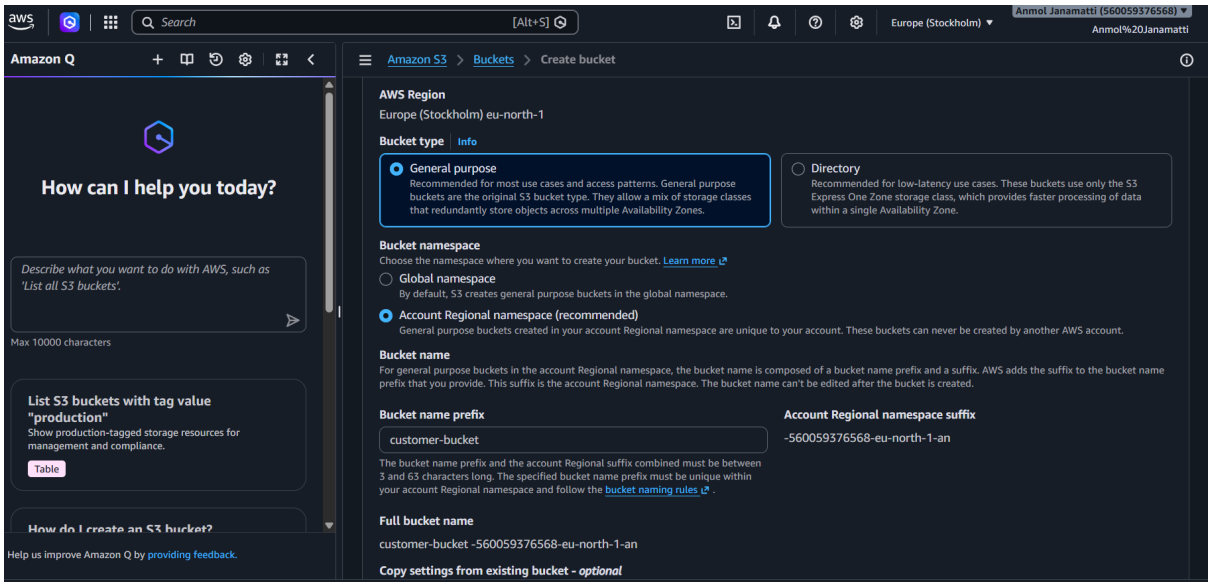
Answer:

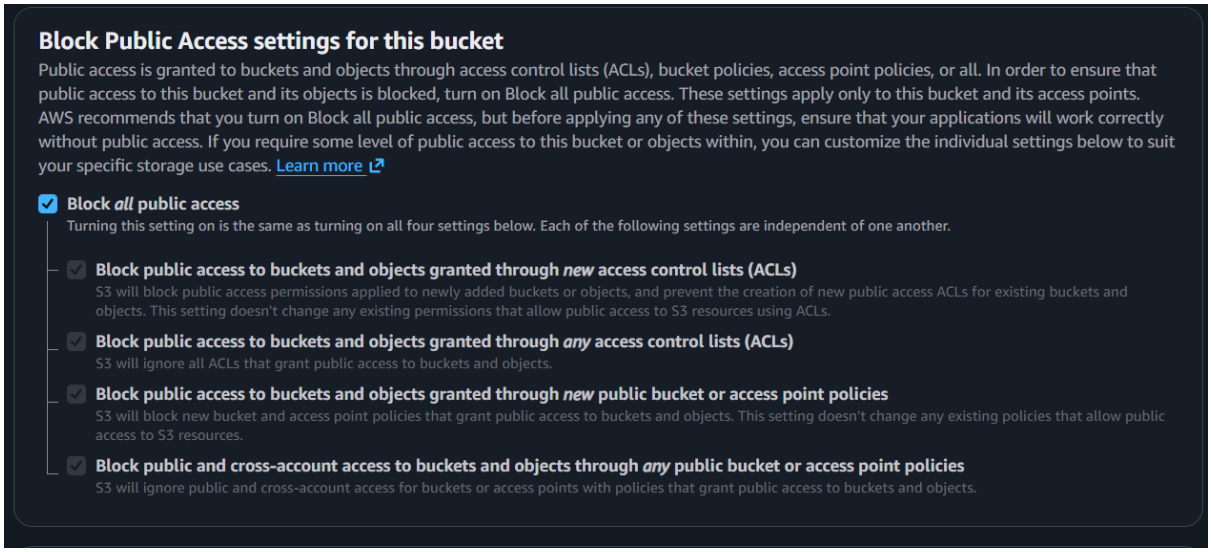
- **Transformations:** Create a new DataFrame or execution plan. Example: filter().
- **Actions:** Execute computations and return results. Example: show().

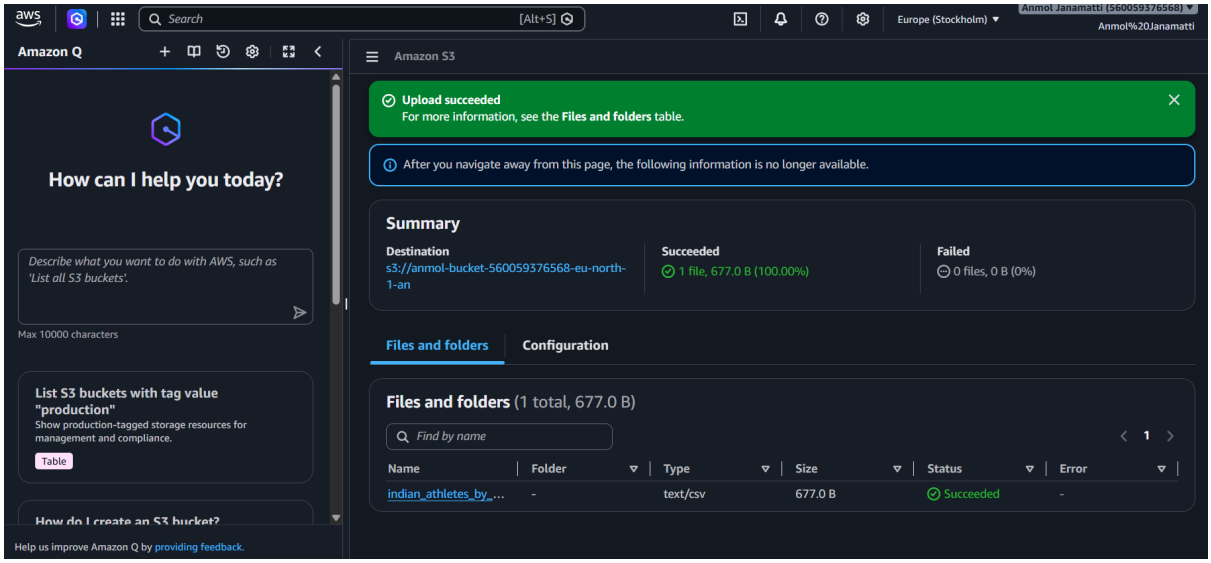
Question 10: S3 and IAM Security

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.

Answer:

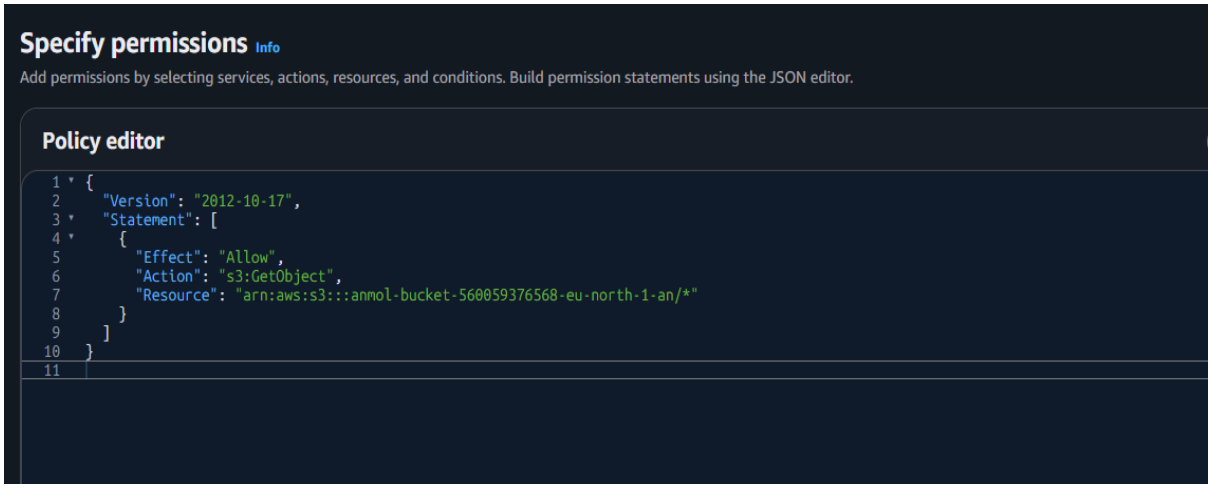






b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.

Answer:



c. Explain how you would verify that an unauthorized user cannot read the file.

Answer:

If an unauthorized user tries to access the file, an Access Denied (403) error will occur. We can verify the failed access attempt using AWS CloudTrail logs (after enabling S3 data events).

d. Explain how encryption at rest protects the stored data.

Answer:

Encryption at rest protects stored data by encrypting it using encryption keys. Even if an unauthorized person gains access to the storage, they cannot read

the data without the required decryption key. In AWS S3, this can be implemented using Server-Side Encryption (SSE).

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.

Answer:

Open the AWS EC2 console and check whether the instance is in the Running state. Verify that both system status checks and instance status checks are passing.

b. Explain how security groups and route tables can affect application connectivity.

Answer:

Security groups control incoming and outgoing traffic to the EC2 instance. Incorrect rules may block application traffic. Route tables determine where network traffic is sent, and incorrect routes can prevent users from connecting to the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

Answer:

Open Amazon CloudWatch and create an alarm for the EC2 instance's CPUUtilization metric. Set the threshold to greater than 85% and define the evaluation period. Configure an SNS notification to alert the administrator when the alarm is triggered.

d. Identify one method for investigating the root cause of high CPU utilization.

Answer:

Use the EC2 instance's operating system tools, such as the top command in Linux, to identify processes consuming high CPU resources.

Question 12: Event-Driven Processing

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

a. Identify a suitable service for executing the processing code.

Answer:

AWS Lambda is suitable for executing code to validate and process uploaded CSV files automatically.

b. Explain how an S3 upload can trigger processing.

Answer:

Configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded to the input bucket.

c. Explain the IAM permissions needed to read the input file and write the results.

Answer:

Assign an IAM execution role to Lambda with s3:GetObject permission for the input bucket and s3:PutObject permission for the output bucket.

d. Explain how errors and processing logs can be captured.

Answer:

Use Amazon CloudWatch Logs to store Lambda execution logs and errors. CloudWatch can also be used to monitor failures.

e. Explain how to prevent duplicate processing or recursive triggering.

Answer:

Use a separate S3 bucket for the processed results so that output uploads do not trigger the same Lambda function. To prevent duplicate processing, make the function idempotent by tracking processed files.

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification.

Answer:

Use AWS Step Functions with the following states: Task states for validation, data transformation, sales calculation, S3 storage, and sending the completion

notification. Use Choice to check whether the input file is valid and Fail to handle invalid files.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

Answer:

The Choice state checks the validation result. If the file is valid, it moves to processing. If the file is invalid, it routes the workflow to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

Answer:

Retry automatically retries temporary failures, such as service errors. Catch handles errors that cannot be recovered by retrying and routes the workflow to an error-handling state or Fail state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

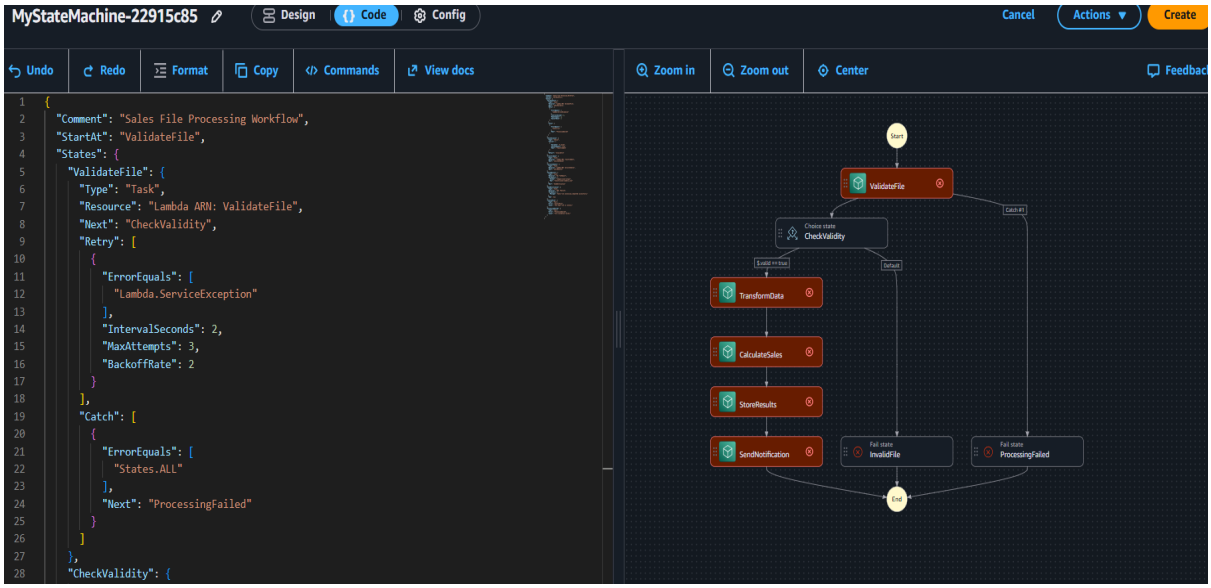
Answer:

Step Functions can invoke AWS Lambda functions using Task states to validate, transform, and calculate sales. It can also use supported AWS service integrations to store results in S3 and send notifications through Amazon SNS.

e. Explain how execution history helps troubleshoot a failed workflow.

Answer:

Step Functions execution history shows each state's input, output, status, and error details. This helps identify the failed state and troubleshoot the problem.



Question 14: Step Functions Implementation

A data engineering team has developed three processing functions:

• ValidateFile

• TransformData

• LoadResults

The functions must execute in sequence. Requirements:

• If validation fails, the workflow must stop.

• If transformation fails temporarily, the workflow must retry.

• If loading fails permanently, the workflow must record the failure.

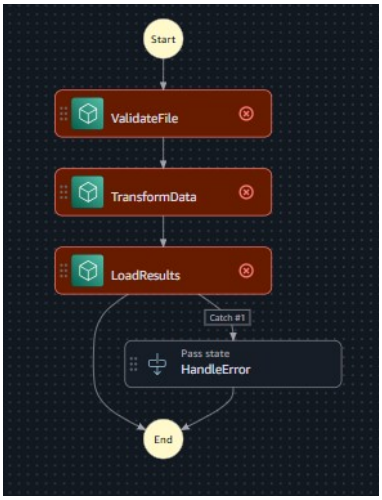
Tasks:

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

Answer:



```
{
  "Comment": "Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "Lambda ARN: ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "Lambda ARN: TransformData",
      "Retry": [
        {
          "ErrorEquals": [
            "Lambda.ServiceException"
          ],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "Lambda ARN: LoadResults",
      "Catch": [
        {
          "ErrorEquals": [
            "States.ALL"
          ],
          "Next": "HandleError"
        }
      ],
      "End": true
    },
    "HandleError": {
      "Type": "Pass",
      "Result": "Loading failed",
      "End": true
    }
  }
}
```



b. Configure a retry policy for the transformation task.

Answer:

The Retry state retries temporary errors such as Lambda.ServiceException. It waits for 2 seconds, retries up to 3 times, and doubles the wait time after each retry using BackoffRate: 2.

c. Configure a Catch transition for the loading task.

Answer:

The Catch block captures loading errors and transfers control to the HandleError state, where the failure is recorded.

d. Explain how data from one task is passed to the next task.

Answer:

Step Functions passes the output of one task as the input to the next task by default. JSON data can be used to share processing results between functions.

e. Explain how you would test successful and failed executions.

Answer:

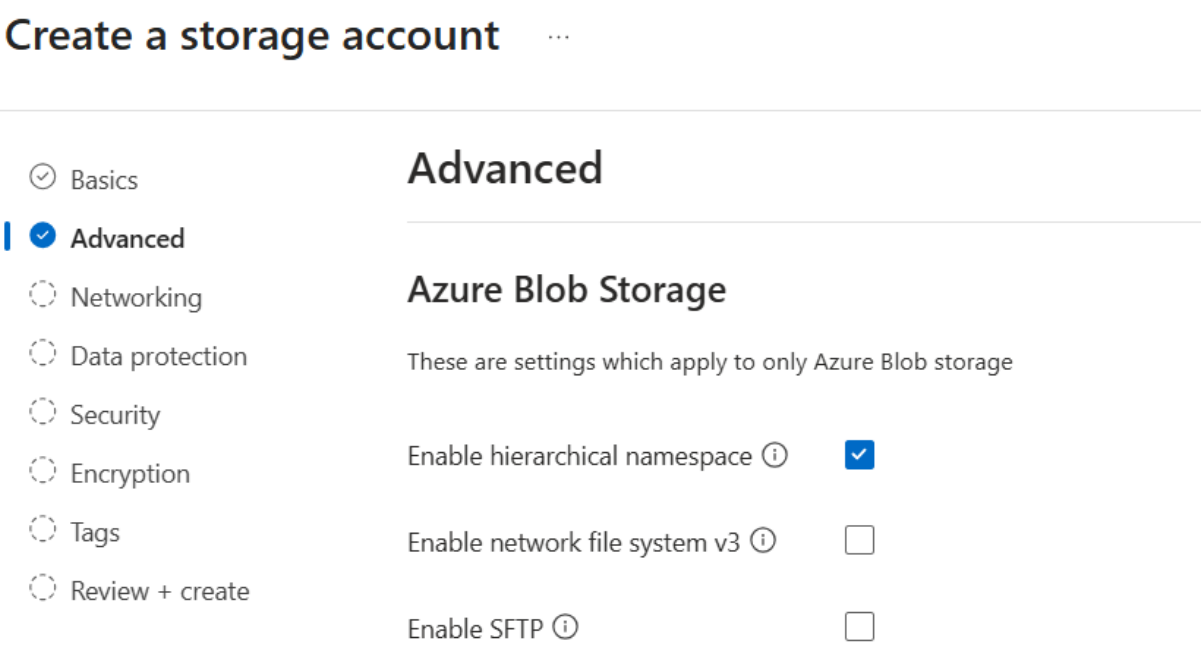
Run the workflow with valid input to verify successful completion. Test with invalid input and simulated transformation or loading errors to verify that retries and error handling work correctly.

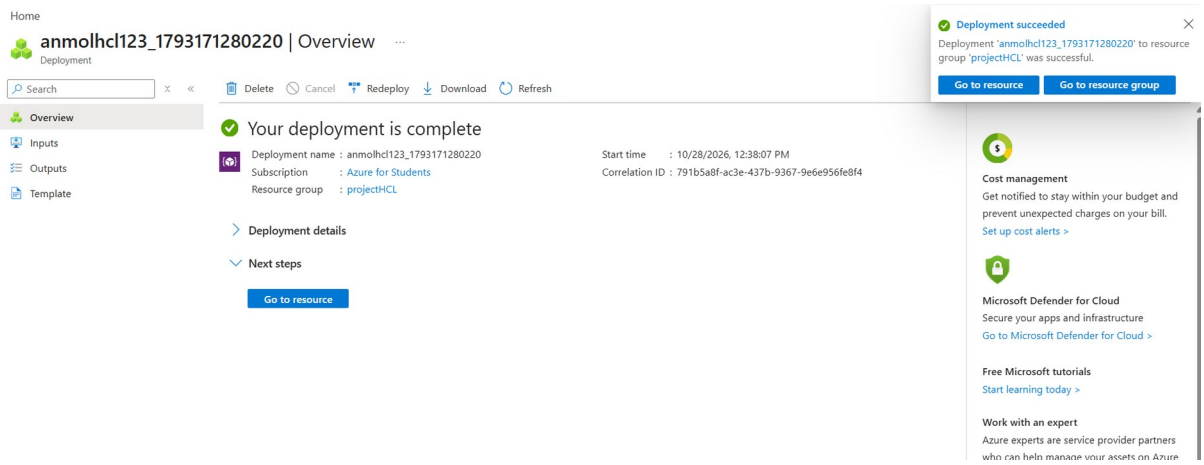
Question 15: Azure Data Lake Storage and Security

a. Configure a storage account with hierarchical namespace enabled.

Answer:

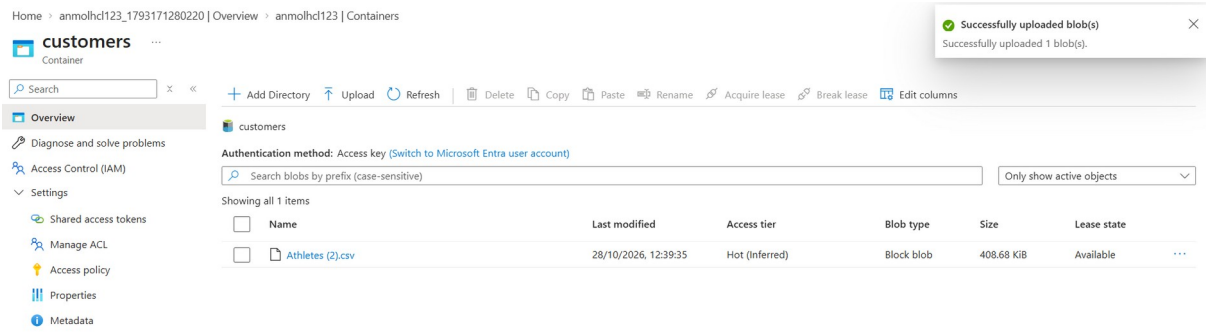
Open the Azure Portal and create a Storage Account. Select the appropriate subscription and resource group, then enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.





b. Create a filesystem or container and upload a sample dataset.

Answer:



Open the storage account, go to Containers, create a container, and upload a sample CSV dataset for analytics.

c. Explain how Microsoft Entra ID and Azure RBAC manage access to the data.

Answer:

Microsoft Entra ID manages user identities and authentication. Azure RBAC assigns permissions to users or groups based on their roles, such as Storage Blob Data Reader or Storage Blob Data Contributor, to control access to the data.

d. Explain how you would verify that unauthorized users cannot access the dataset.

Answer:

Sign in as a user without the required permissions and attempt to access or download the dataset. Verify that access is denied, and check the Azure Monitor or diagnostic logs if configured

Question 16: Azure VM Troubleshooting

A virtual machine is running in Azure, but administrators cannot connect to it.

Perform or document the following:

a. Verify the VM's provisioning and power states.

Answer:

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how Network Security Group rules can prevent connectivity.

Answer:

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Explain how subnet and routing configuration affect connectivity.

Answer:

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

Answer:

Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.

Question 17: Azure Data Integration

A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement.

Answer:

Azure Data Factory (ADF) is the most suitable service. Its pipelines, triggers, and Copy activity orchestrate data movement from the source to Azure Data Lake Storage Gen2 and can run the load every day.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2.

Answer:

Create linked services for the source and ADLS Gen2 (using a managed identity with the Storage Blob Data Contributor role) and define DelimitedText datasets. Use a Copy activity in an ADF pipeline to copy the CSV file into a data lake container, started by a daily schedule trigger.

c. Describe how to organize files by ingestion date.

Answer:

Use a dynamic folder path in the sink dataset, such as raw/transactions/yyyy/MM/dd/, built from the trigger time with formatDateTime(). This keeps each day's file in its own date folder and makes reprocessing and date-based queries easier.

d. Explain how to handle malformed records and missing values.

Answer:

Use a Mapping Data Flow to validate records. Route rows with malformed values or missing required fields (such as TransactionID or CustomerID) to a rejected folder using a Conditional Split, and fill non-critical missing values with defaults using coalesce(). Alternatively, enable fault tolerance in the Copy activity to skip and log incompatible rows.

e. Explain how to monitor pipeline execution and validate data completeness.

Answer:

Use the Monitor tab in Azure Data Factory to track pipeline and activity runs, and configure Azure Monitor alerts for failures. Validate completeness by comparing the rows read, rows written, and rows rejected in the Copy activity output with the source record count.